ShortP2P Protocol

Sprache: German / Deutsch

Author: Igor Evdokimov

Repository: https://github.com/IskraMessenger/ShortP2P

Created: 2026-11-11

Modified: 2026-11-11

Related: docs/encryption, docs/messages, protocol.md

Inhalt

Titelseite

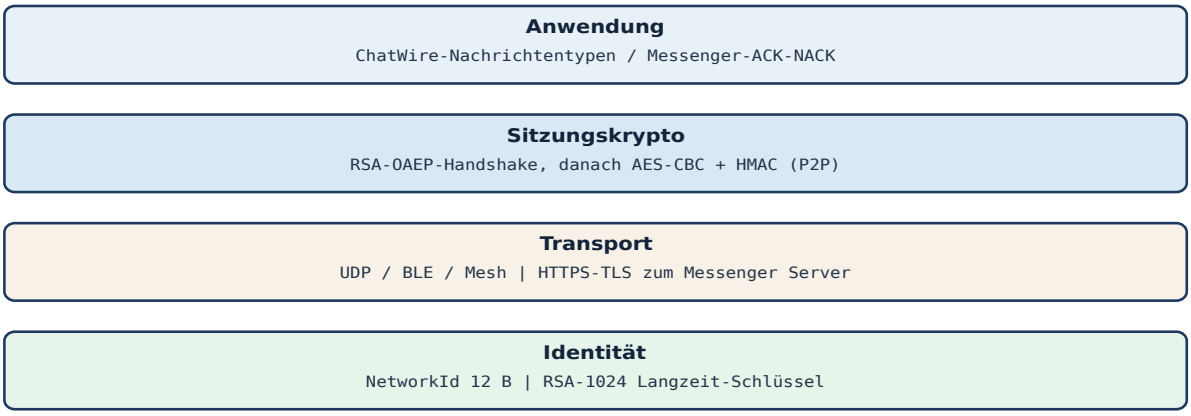
Inhalt

Gemeinsame Diagramme

Protokollbeschreibung

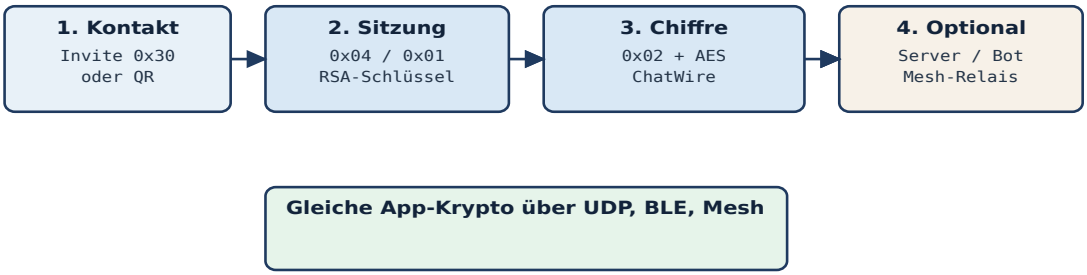
Gemeinsame Diagramme

Protokoll-Schichten

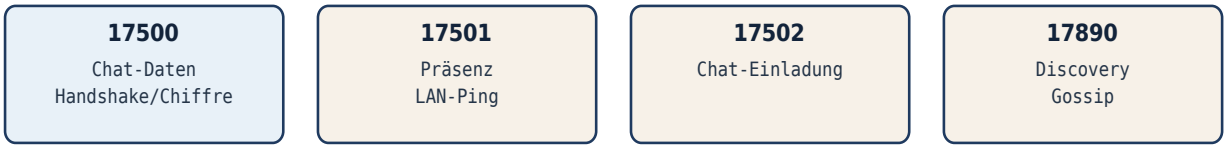


Logischer Stack (keine OS-Schichten). Details: docs/encryption, docs/messages.

Typischer direkter Chat-Ablauf



Standard-UDP-Ports



Bluetooth und Mesh nutzen dieselben Frames; HTTPS für Messenger Server.

Protokollbeschreibung

ShortP2P ist ein Peer-to-Peer-Messaging-Stack. Einfach gesagt: Zwei Geräte (Peers) finden einander, vereinbaren geheime Schlüssel und tauschen dann Nachrichten aus, die dazwischen niemand lesen kann. Der Transport kann direkt sein (UDP, Bluetooth, Mesh/LoRa-Funk) oder über einen optionalen Messenger Server laufen, der nur bereits verschlüsselte Daten weiterleitet.

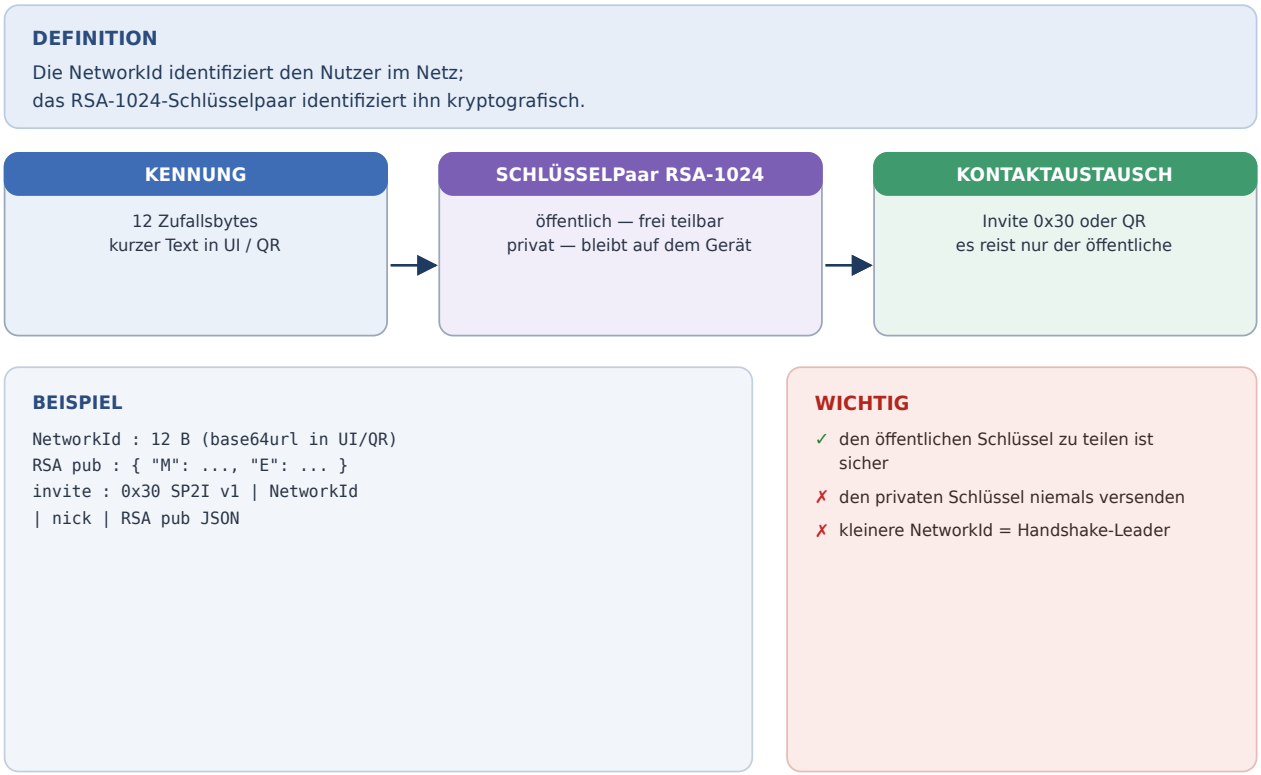
GRUNDBEGRIFFE (keine Krypto-Vorkenntnisse nötig)

- Peer: das andere Gerät / die Person, mit der man schreibt.
- NetworkId: 12 Zufallsbytes, die einen Nutzer identifizieren — wie eine Netzwerkadresse. In UI und QR-Codes als kurzer base64url-Text (~16 Zeichen).
- RSA-1024: ein asymmetrisches Verfahren — ein Schlüsselpaar. Der öffentliche Schlüssel wird offen geteilt (in Einladungen und QR-Codes); der private verlässt das Gerät nie. Was mit dem öffentlichen Schlüssel verschlüsselt wurde, kann nur mit dem zugehörigen privaten Schlüssel entschlüsselt werden. In ShortP2P wird dies einmal pro Sitzung benutzt, um dem Peer die Sitzungsgeheimnisse zu senden.
- Sitzungsschlüssel: zwei zufällige Byteketten, erzeugt für eine Chat-Sitzung: ein AES-Schlüssel (16 Bytes) und ein MAC-Schlüssel (32 Bytes). Danach besitzen beide Seiten dieselben zwei Ketten.
- AES-128-CBC: eine symmetrische Verschlüsselung — derselbe Schlüssel ver- und entschlüsselt. Der CBC-Modus verkettet Blöcke, damit identische Textstücke unterschiedlich aussehen; PKCS7-Padding richtet die Länge auf ein Vielfaches von 16 Bytes aus.
- IV (Initialisierungsvektor): 16 frische Zufallsbytes pro Paket, damit derselbe Text bei jedem Senden anders verschlüsselt aussieht.
- HMAC-SHA256-Tag: eine schlüsselgestützte Prüfsumme (16 Bytes werden verwendet) über IV + Geheimtext. Der Empfänger rechnet sie neu und vergleicht in konstanter Zeit; eine Abweichung heißt: Paket verändert oder beschädigt — es wird verworfen.
- ChatWire: ShortP2Ps eigenes Binärformat für Chatinhalte. Genau das wird verschlüsselt und gesendet; der Empfänger entschlüsselt zuerst und zerlegt es danach.
- S2P1: eine 4-Byte-„magische Signatur“ (ASCII "S2P1") am Anfang jedes ChatWire-Frames. Sie wirkt wie ein Datei-Header — das Erste, was der Parser prüft, um das Format zu erkennen.
- Kind-Byte: das Byte direkt nach der Magie. Sein Wert sagt, welcher Nachrichtentyp folgt.
- ACK / NACK: interne Zustellberichte. ACK = „Nachricht vollständig empfangen“; NACK = „bitte die Datenblöcke mit diesen Nummern erneut senden“.
- TLS/HTTPS: die übliche Transportverschlüsselung zwischen Client und Messenger Server (das Schloss im Browser).
- JWT: ein signiertes Token, das der Server beim Login ausstellt. Der Client hängt es an jede Anfrage; der Server prüft die Signatur, statt erneut das Passwort zu verlangen.
- Ende-zu-Ende (E2E): den Inhalt können nur die beiden chattenden Geräte lesen. Der

Messenger Server speichert und leitet ihn als opaken verschlüsselten Block (Base64-Text) weiter und kann ihn selbst nicht lesen.

- Bot: ein Programm-Account auf dem Server, mit dem Clients reden können; auch er erhält nur verschlüsselten Inhalt.
- Gossip / Discovery: Hilfsverkehr, der „wer ist in der Nähe und wie erreiche ich ihn“ beantwortet (UDP 17890).
- RELAY: ein LAN-Routing-Frame, der bereits verschlüsselte Nutzdaten über Zwischenknoten tragen kann.

1) IDENTITÄT



Jeder Nutzer hat eine NetworkId (12 Zufallsbytes) und ein bei der Registrierung erzeugtes RSA-1024-Schlüsselpaar. Der öffentliche Schlüssel reist im Klartext in Chat-Einladungen (0x30) oder QR-Codes — das ist unkritisch, denn ein öffentlicher Schlüssel ist zum Teilen da. Regel: Der Peer mit der kleineren NetworkId wird „Leader“ des Handshakes; die andere Seite wartet.

2) TRANSPORTE

DEFINITION

Dieselben logischen Frames laufen über mehrere Transporte;
nur die Verbindung zum Messenger Server ist immer HTTPS.

UDP 17500	UDP 17501/17502	UDP 17890	BLE / MESH
Datenkanal des Chats Handshake + Chiffre 0x02	Präsenz / Einladungen Klartext-Steuerung	Discovery / Gossip Klartext-Steuerung	gleiche logische Frames falls aktiviert

BEISPIEL

```
0x30 invite | 0x31 presence
0x01 handshake | 0x04 setup req
0x02 cipher | discovery: 17890
BLE / mesh: same logical frames
server: HTTPS/TLS + JWT always
```

WICHTIG

- ✓ Chatinhalt nur innerhalb der Chiffre 0x02
- ✗ Invite / Präsenz / Gossip sind offen
- ✗ Serververbindung immer HTTPS + JWT

UDP-Ports standardmäßig: 17500 — Datenkanal des Chats (Handshake und verschlüsselte Pakete), 17501 — Anwesenheitsmeldungen, 17502 — Einladungen, 17890 — Discovery/Gossip. Bluetooth und Mesh/LoRa tragen dieselben logischen Frames, wenn aktiviert. Die Verbindung zum Messenger Server läuft immer über HTTPS/TLS mit JWT-Authentifizierung.

3) SITZUNG (direktes P2P), Schritt für Schritt

DEFINITION

Der Leader sendet zwei Zufallsgeheimnisse im öffentlichen Schlüssel des Peers;
danach verschlüsseln beide Seiten mit demselben AES-Schlüssel.

ANFRAGE 0x04	48 ZUFALLSBYTES	RSA-OAEP 0x01	GEMEINSAME GEHEIMNISSE
Follower: „Ich möchte eine Sitzung“	AES-Schlüssel 16 B + MAC 32 B	im öffentlichen Schlüssel genau 128 B	beide Seiten, dieselben Schlüssel

BEISPIEL

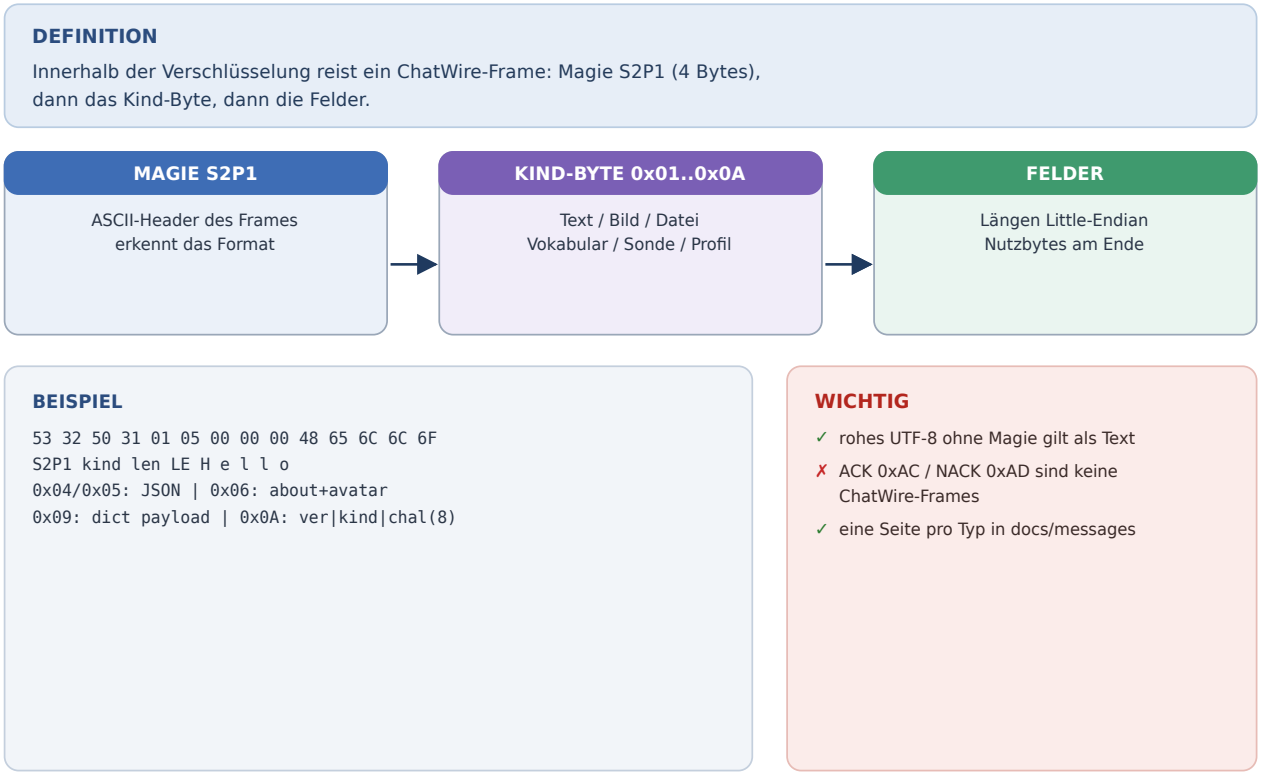
```
0x04 || NetworkId(12)
0x01 || RSA-OAEP-SHA1(128) // keys 48 B
0x02 || IV(16) || AES-CBC || tag(16)
packet <= 1024 B | plain <= 991 B
```

WICHTIG

- ✓ erst das HMAC-Tag prüfen, dann entschlüsseln
- ✗ der Leader ignoriert fremde 0x01
- ✗ IV pro Paket frisch zufällig

a. Der Follower kann Frame 0x04 mit seiner NetworkId senden: „Ich möchte eine Sitzung“.
b. Der Leader erzeugt zwei Zufallsgeheimnisse (AES-Schlüssel 16 B + MAC-Schlüssel 32 B) und verschlüsselt dieses 48-Byte-Bündel mit dem öffentlichen RSA-Schlüssel des Peers per RSA-OAEP(SHA-1). Das Ergebnis ist genau 128 Bytes und wird als Frame 0x01 gesendet.
c. Der Follower entschlüsselt mit seinem privaten RSA-Schlüssel; danach haben beide Seiten dieselben Geheimnisse.
d. Jeder weitere Datenframe ist 0x02 + Paket, wobei Paket = IV(16) + AES-CBC-Geheimtext + HMAC-Tag(16), höchstens 1024 Bytes. Der Empfänger prüft zuerst das Tag, dann entschlüsselt er.

4) ANWENDUNGSNACHRICHTEN (innerhalb der Verschlüsselung)



Der Klartext, der in die Verschlüsselung geht, ist meist ein ChatWire-Frame: Magie "S2P1", dann das Kind-Byte, dann Felder (Längen Little-Endian, sofern nicht anders angegeben).

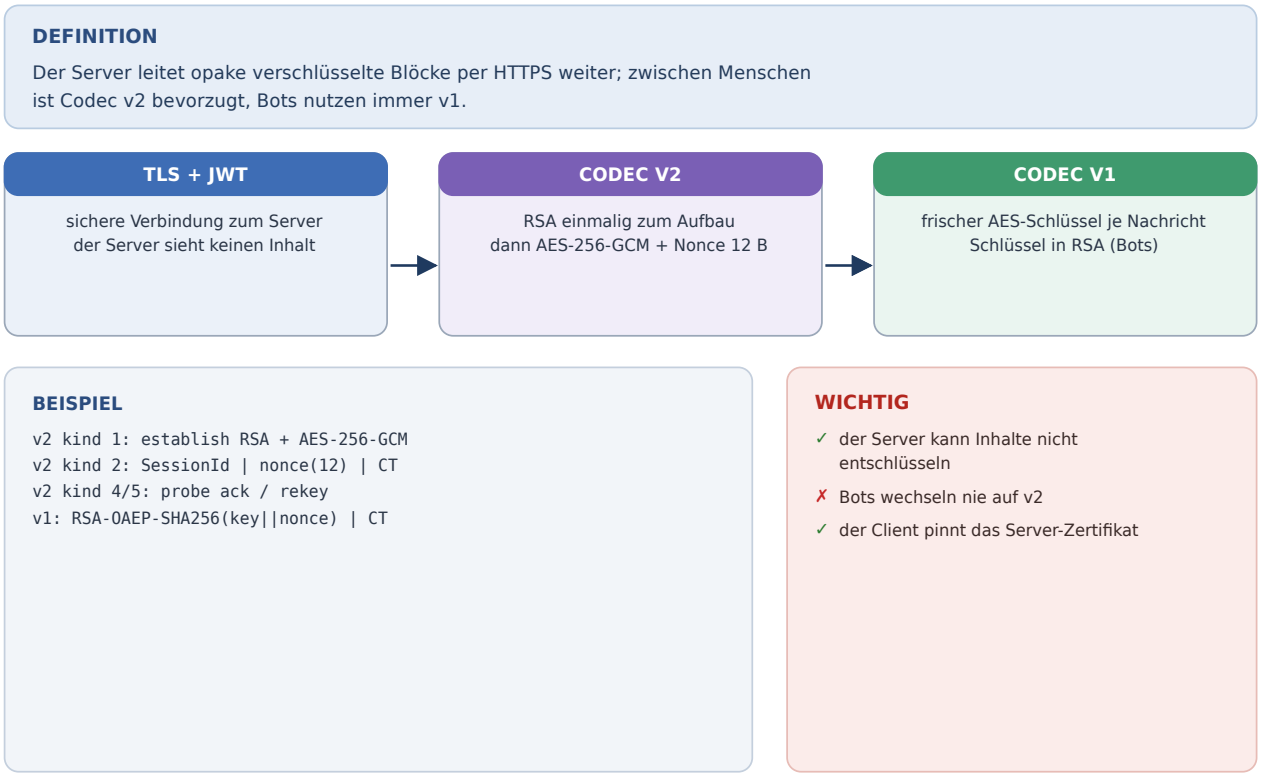
Die Kind-Byte-Werte, ausgeführt:
- 0x01 Text — eine normale Textnachricht (UTF-8), was ein Mensch eintippt.
- 0x02 Image — ein Bild im Chat: MIME-Typ (z. B. "image/png") plus die Bilddaten.
- 0x03 File — eine kleine Datei im Chat: Name, MIME-Typ, Bytes.
- 0x04 Transfer offer — eine JSON-Beschreibung einer großen Datei, die außerhalb des Chats zum Download angeboten wird (Host/Port, Einmal-Token, Größe, Ablauf).
- 0x05 Transfer control — die JSON-Antwort auf ein Angebot: annehmen, ablehnen oder Fehler.

- 0x06 User info — das Profil des Absenders („Über mich“-Text und Avatar), nicht als Chat-Bubble sichtbar.
- 0x07 Vocabulary offer — die Liste der Kompressionswörterbücher, die der Absender beherrscht.
- 0x08 Vocabulary accept — die Wahl der Wörterbücher für diesen Chat (oder "UNCOMPRESSED").
- 0x09 Compressed text — ein mit dem gewählten Wörterbuch kodierter Text, um Traffic zu sparen (nützlich auf langsamen Funkstrecken).
- 0x0A Session probe — ein kleiner „Ping“ durch den verschlüsselten Kanal, um zu bestätigen, dass beide Seiten korrekt entschlüsseln.

Keine ChatWire-Frames, aber ebenfalls verschlüsselt: 0xAC ACK (Nachricht vollständig empfangen, 17 Bytes) und 0xAD NACK (bitte die gelisteten Block-Nummern erneut senden). Sehr alte Clients senden rohes UTF-8 ohne die S2P1-Magie; der Empfänger behandelt es trotzdem als Text.

Eine Seite pro Typ mit Byte-Layouts und unverschlüsselten Beispielen: docs/messages. Krypto-Pipelines als Diagramme: docs/encryption.

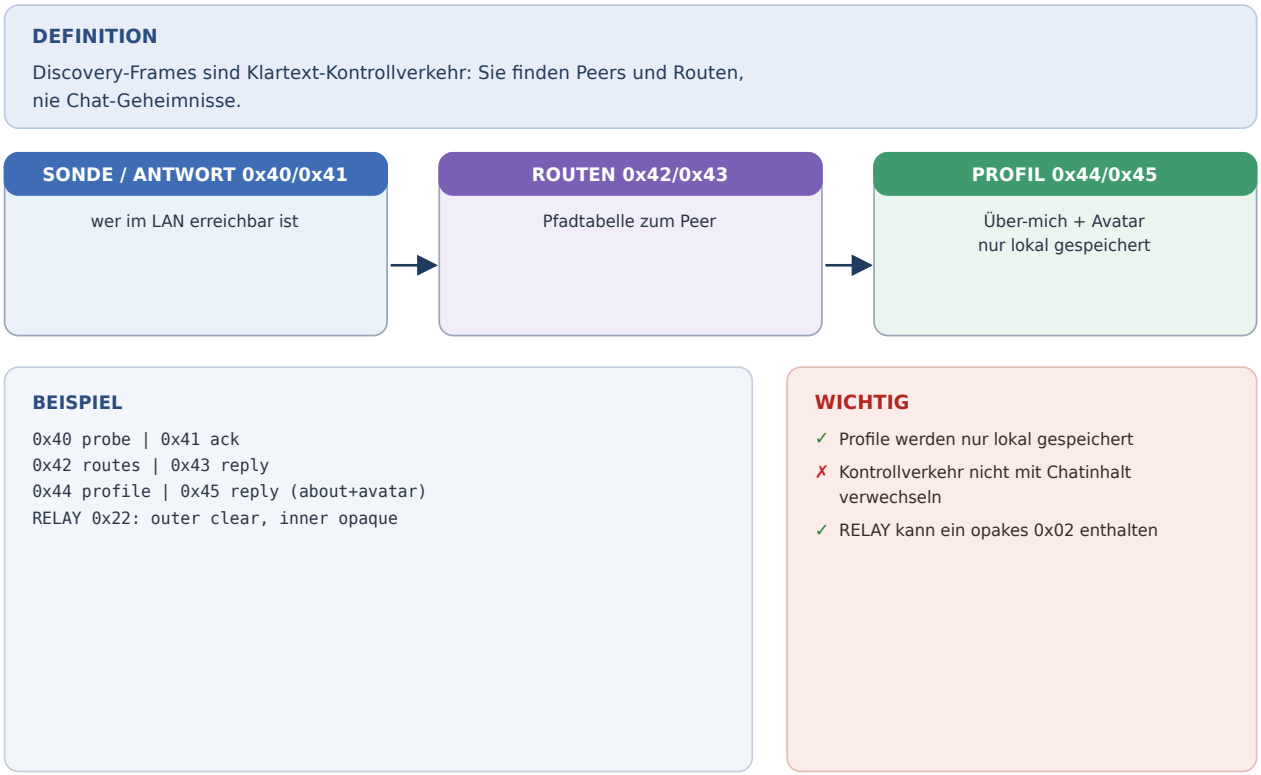
5) MESSENGER SERVER UND BOTS



Wenn Peers sich nicht direkt verbinden können, läuft es über den Messenger Server per HTTPS/TLS+JWT. Der Chatinhalt ist Ende-zu-Ende-verschlüsselt: Der Server speichert nur einen opaken Base64-Block.
- Codec v2 (zwischen Menschen bevorzugt): RSA wird einmalig zum Sitzungsaufbau genutzt; danach ist jede Nachricht AES-256-GCM (eine authentisierte Verschlüsselung:

Verschlüsselung und Integritätsprüfung in einem Algorithmus) mit einer frischen 12-Byte-Zufalls-Nonce pro Nachricht.
- Codec v1 (für Bots und als Fallback): jede Nachricht erhält einen frischen AES-256-GCM-Schlüssel und eine Nonce; Schlüssel+Nonce werden mit dem öffentlichen RSA-Schlüssel des Empfängers eingepackt (RSA-OAEP-SHA256). Bots nutzen immer v1.
In beiden Fällen kann der Server den Inhalt nicht entschlüsseln.

6) DISCOVERY



Frames 0x40..0x45 auf UDP 17890 (Sonnen/Antworten, Routentabellen, Profile) sind meist Klartext-Kontrollverkehr: Sie helfen, Peers zu finden, und dürfen nicht mit geheimem Chatinhalt verwechselt werden. LAN-FIND/FOUND-Frames ermitteln die Adresse eines Peers; ein RELAY-Frame kann bereits verschlüsselte 0x02-Daten über Zwischenknoten tragen.

WEITERFÜHRENDES

- docs/encryption — Krypto-Pipelines als Vektordiagramme (Sitzungsaufbau, Paketformate, Serverpfade).
- docs/messages — eine Seite pro Nachrichtentyp mit unverschlüsselten Byte-Beispielen.
- protocol.md — detaillierte Frame-Referenz (auf Russisch).